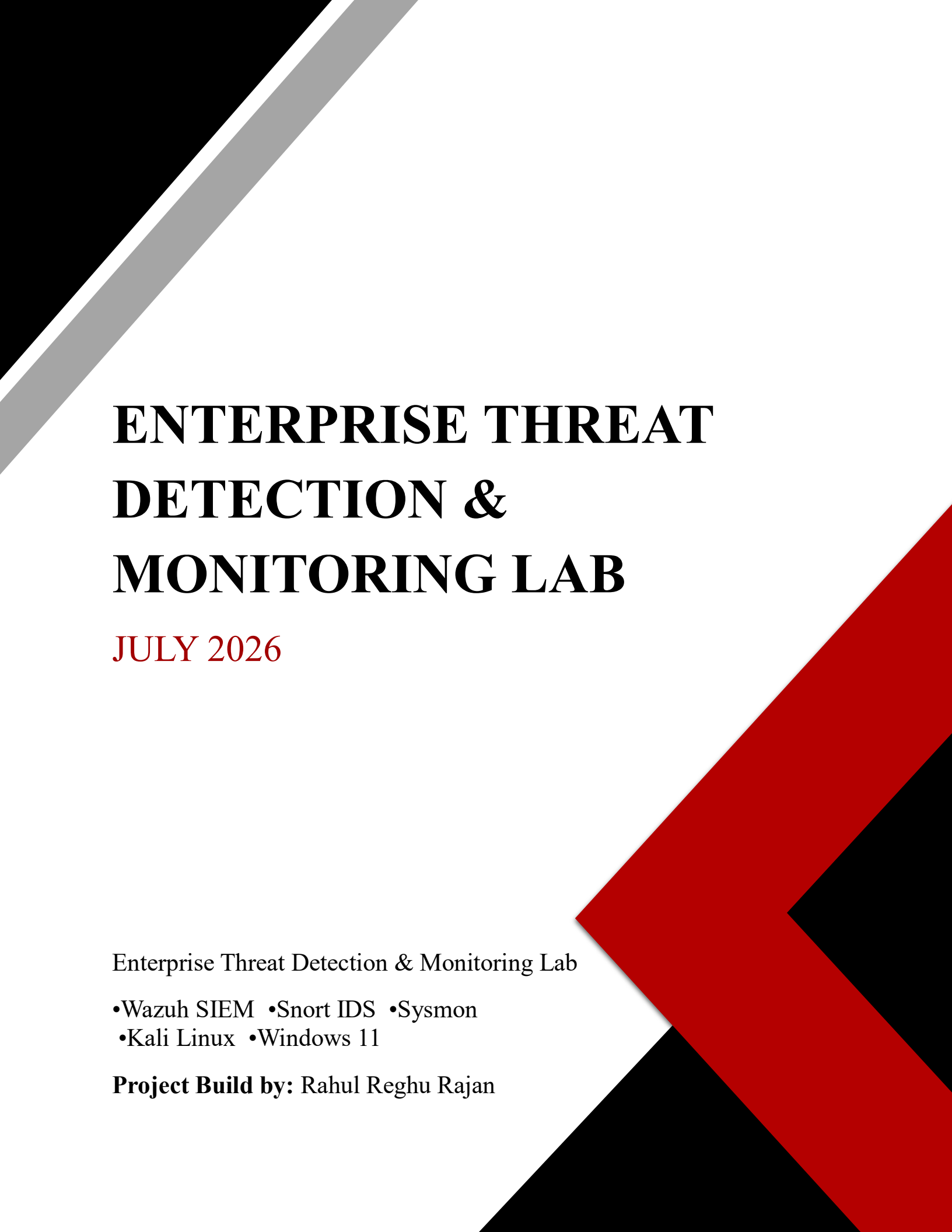




ENTERPRISE THREAT DETECTION & MONITORING LAB

JULY 2026

Enterprise Threat Detection & Monitoring Lab

•Wazuh SIEM •Snort IDS •Sysmon
•Kali Linux •Windows 11

Project Build by: Rahul Reghu Rajan

DISCLAIMER

This report has been developed solely for educational, research, and professional portfolio purposes to demonstrate the implementation of an Enterprise Threat Detection & Monitoring Laboratory. All attack simulations, security monitoring activities, and forensic investigations were performed within a controlled virtual environment owned and managed by the author. No public, third-party, or production systems were targeted during the execution of this project. The techniques demonstrated, including reconnaissance detection, intrusion monitoring, and authentication testing, are standard cybersecurity practices used to evaluate and strengthen defensive security controls. This documentation is intended to showcase practical cybersecurity skills while promoting responsible and ethical use of security tools and methodologies.

CONFIDENTIALITY NOTICE

This document contains technical information regarding the design, implementation, and analysis of a personal cybersecurity laboratory environment. It is intended exclusively for academic evaluation, technical interviews, portfolio presentation, and cybersecurity learning. Any IP addresses, usernames, passwords, virtual machines, screenshots, and network configurations included within this report belong solely to the isolated laboratory environment created for this project and do not represent any real organizational infrastructure. Redistribution, reproduction, or commercial use of this document without prior permission from the author is discouraged. The information presented is provided for educational purposes and follows responsible cybersecurity and ethical testing practices.

TABLE OF CONTENTS

1. Introduction.....	01
2. Executive Summary.....	02
3. Project Objectives.....	03
4. Lab Architecture.....	04
5. Technologies Overview.....	06
6. System Implementation	
6.1 Installing Wazuh SIEM & Agent.....	08
6.2 Installing Sysmon & Integrating with Wazuh.....	12
6.3 Installing Snort IDS & Integrating with Wazuh.....	16
7. Attack Simulations	
7.1 ICMP Reconnaissance Detection.....	21
7.2 TCP SYN / Nmap Port Scan Detection.....	24
7.3 SSH Brute Force Detection (Hydra).....	27
8. Conclusion.....	30
9. Appendix A – Author Information.....	31
10. References.....	33

1. INTRODUCTION

The increasing sophistication of cyber threats has made continuous network monitoring and threat detection an essential component of modern cybersecurity. Organizations rely on Security Information and Event Management (SIEM) platforms, Intrusion Detection Systems (IDS), and endpoint monitoring solutions to identify malicious activities, investigate security incidents, and strengthen their overall security posture. Building hands-on experience with these technologies is crucial for understanding how real-world Security Operations Centers (SOCs) detect, analyze, and respond to potential threats.

This project presents the design and implementation of an **Enterprise Threat Detection & Monitoring Laboratory** using **Wazuh SIEM, Snort IDS, Sysmon, Kali Linux, and Windows 11** within a controlled virtual environment. The laboratory simulates common cyberattack scenarios while demonstrating how security events are collected, monitored, correlated, and investigated through centralized security monitoring. By integrating both network-based and host-based security solutions, the project provides practical insight into enterprise threat detection workflows and defensive security operations.

The primary objective of this project is to demonstrate the complete lifecycle of threat detection from attack simulation and event generation to alert analysis and security monitoring using industry-recognized open-source security tools. This implementation reflects fundamental concepts used in enterprise SOC environments and serves as a practical demonstration of cybersecurity monitoring, incident detection, and log analysis skills.

2. EXECUTIVE SUMMARY

This project presents the development of an Enterprise Threat Detection & Monitoring Laboratory designed to simulate real-world Security Operations Center (SOC) monitoring within a controlled virtual environment. The implementation combines Wazuh SIEM, Snort Intrusion Detection System (IDS), and Sysmon to provide comprehensive visibility into both network traffic and endpoint activities.

The laboratory consists of a Kali Linux attacker machine, a Windows 11 victim machine, and an Ubuntu Server hosting the Wazuh platform. Attack simulations including ICMP reconnaissance, TCP SYN/Nmap port scanning, and SSH brute-force attempts using Hydra were executed from the attacker machine. Snort detected suspicious network activities, while Wazuh centralized, correlated, and visualized the generated security events. Sysmon further enhanced endpoint visibility by collecting detailed Windows system activities beyond standard event logging.

The project demonstrates the complete threat detection lifecycle, including attack simulation, event collection, centralized log management, alert investigation, and security monitoring. By integrating multiple open-source security technologies into a single monitoring platform, this laboratory provides practical experience with enterprise-level detection engineering, incident investigation, and defensive cybersecurity operations commonly performed within modern Security Operations Centers.

3. PROJECT OBJECTIVES

The primary objective of this project is to design and implement an Enterprise Threat Detection & Monitoring Laboratory capable of simulating real-world cyberattack scenarios while demonstrating effective threat detection, centralized log management, and security event monitoring using open-source security technologies.

The project was developed to achieve the following objectives:

- Design a practical Security Operations Center (SOC) laboratory using virtualized infrastructure.
- Deploy and configure Wazuh SIEM as the centralized platform for log collection, correlation, and security monitoring.
- Integrate Snort IDS with Wazuh to detect and monitor malicious network activities.
- Integrate Microsoft Sysmon with Wazuh to provide detailed endpoint visibility beyond standard Windows Event Logs.
- Simulate real-world cyberattack scenarios including ICMP reconnaissance, TCP SYN/Nmap port scanning, and SSH brute-force attacks using Hydra.
- Investigate generated alerts within the Wazuh dashboard and analyze attack indicators from both endpoint and network perspectives.
- Demonstrate the complete threat detection lifecycle, including attack simulation, detection, event correlation, investigation, and security monitoring.
- Develop practical skills in Security Information and Event Management (SIEM), Intrusion Detection Systems (IDS), endpoint monitoring, and enterprise threat analysis.

4. LAB ARCHITECTURE

The Enterprise Threat Detection & Monitoring Laboratory was designed to simulate a simplified enterprise Security Operations Center (SOC) environment using virtualized infrastructure and open-source security technologies. The architecture consists of an attacker system, a monitored endpoint, a centralized Security Information and Event Management (SIEM) platform, and an Intrusion Detection System (IDS), all working together to detect, collect, correlate, and analyze security events.

The laboratory uses a **Kali Linux** virtual machine as the attacker to perform controlled cyberattack simulations, including ICMP reconnaissance, TCP SYN/Nmap port scanning, and SSH brute-force attacks. These attacks generate network traffic similar to real-world adversarial activity.

The **Windows 11** virtual machine acts as the monitored endpoint. Microsoft **Sysmon** is installed on the endpoint to capture detailed system activities such as process creation, network connections, registry modifications, file events, and other security-relevant operations. In addition, Windows Event Logs provide operating system and authentication events, which are forwarded to the SIEM for centralized monitoring.

A dedicated **Ubuntu Server** hosts the **Wazuh SIEM** platform, which serves as the central monitoring and analysis system. Wazuh collects logs from the Windows endpoint, correlates security events, applies detection rules, and presents alerts through a web-based dashboard for investigation.

To provide network-level visibility, **Snort IDS** is deployed on the Windows endpoint. Snort monitors live network traffic and detects suspicious activities such as reconnaissance attempts, abnormal TCP behavior, and other potentially malicious packets. The generated IDS alerts are forwarded to Wazuh, enabling centralized investigation alongside endpoint telemetry.

Together, these components simulate the complete enterprise threat detection workflow, beginning with attack execution, followed by network and endpoint monitoring, centralized log collection, event correlation, alert generation, and security investigation through a single monitoring platform.

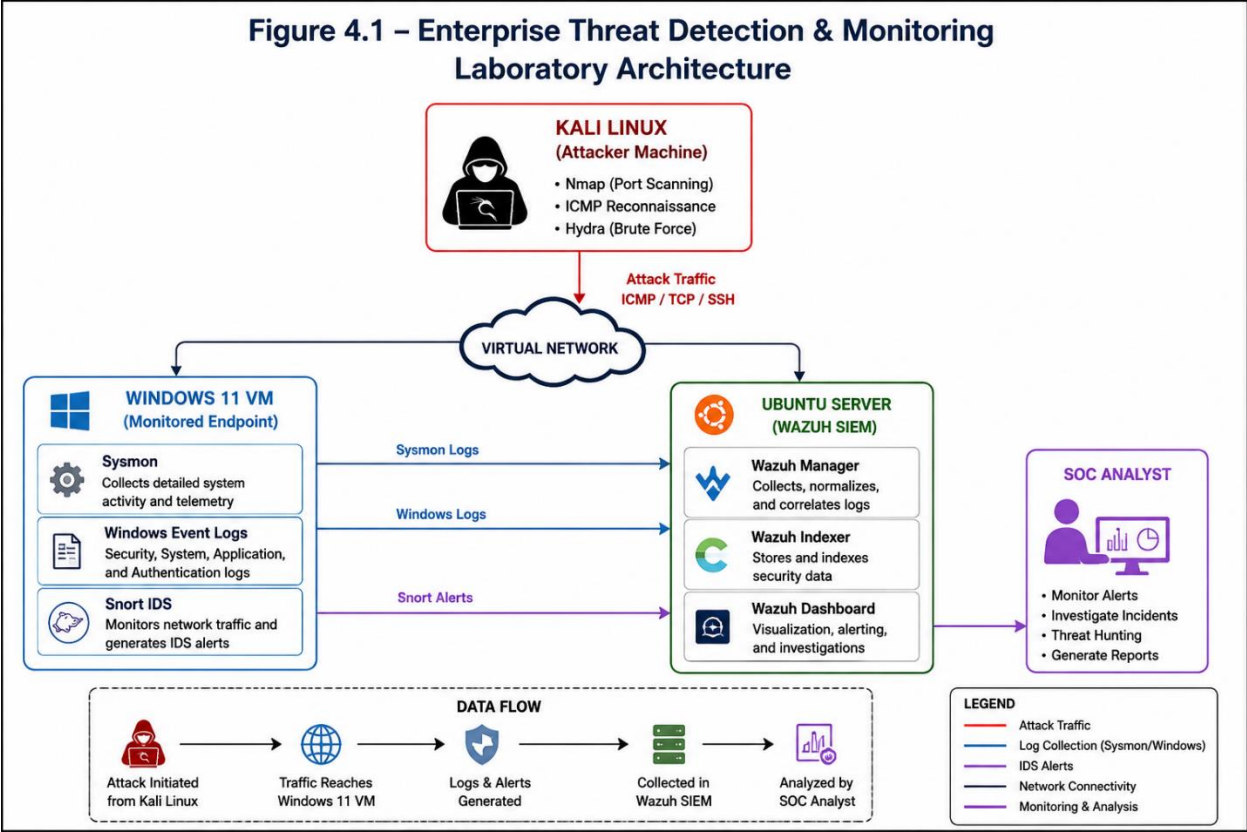


Figure 4.1 – Enterprise Threat Detection & Monitoring Laboratory Architecture

5. TECHNOLOGIES OVERVIEW

This Enterprise Threat Detection & Monitoring Laboratory integrates multiple open-source cybersecurity technologies to simulate a modern Security Operations Center (SOC) environment. Each technology performs a specific role in detecting, collecting, correlating, and investigating security events generated during attack simulations.

Kali Linux

Kali Linux serves as the attacker machine used to perform controlled security assessments against the target system. Tools such as Nmap, Hydra, and standard ICMP utilities were used to simulate reconnaissance, port scanning, and brute-force authentication attacks.

Windows 11

Windows 11 functions as the monitored endpoint within the laboratory. It represents an enterprise workstation that generates system logs, network traffic, authentication events, and security-related activities during the attack simulations.

Wazuh SIEM

Wazuh acts as the centralized Security Information and Event Management (SIEM) platform. It collects security events from multiple sources, normalizes logs, correlates related events, applies detection rules, and provides a centralized dashboard for monitoring and incident investigation.

Wazuh Agent

The Wazuh Agent is installed on the Windows endpoint to continuously monitor operating system activities and securely forward logs to the Wazuh Manager for analysis.

Sysmon

Microsoft Sysmon extends Windows logging by capturing detailed endpoint telemetry, including process creation, network connections, driver loading, registry

modifications, and other security-relevant activities that are not available through standard Windows Event Logs.

Snort IDS

Snort is deployed as the network-based Intrusion Detection System (IDS). It inspects network packets in real time and generates alerts whenever suspicious traffic patterns or attack signatures are detected. The generated alerts are forwarded to Wazuh for centralized investigation.

Windows Event Logs

Windows Event Logs provide native operating system events related to authentication, security, applications, and system operations. These logs complement Sysmon by providing additional visibility into endpoint activities.

MITRE ATT&CK Framework

The MITRE ATT&CK Framework is used to classify detected attack techniques according to industry-recognized adversary tactics and techniques. This provides structured threat context during security investigations.

6. SYSTEM IMPLEMENTATION

This section describes the deployment, configuration, and integration of the core security technologies used within the Enterprise Threat Detection & Monitoring Laboratory. Each component was installed, verified, and integrated with Wazuh to enable centralized log collection, endpoint monitoring, and network intrusion detection.

6.1 Installing Wazuh SIEM & Agent

The Wazuh SIEM platform was deployed on an Ubuntu Server using the official all-in-one installation method. After successful installation, the Wazuh Agent was installed on the Windows 11 endpoint and registered with the Wazuh Manager to enable secure log forwarding and centralized security monitoring.

Step 1 – Update the Ubuntu Server

```
sudo apt update
```

```
sudo apt upgrade -y
```

Step 2 – Download and Install Wazuh

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
```

```
sudo bash ./wazuh-install.sh -a
```

Step 3 – Verify Wazuh Services

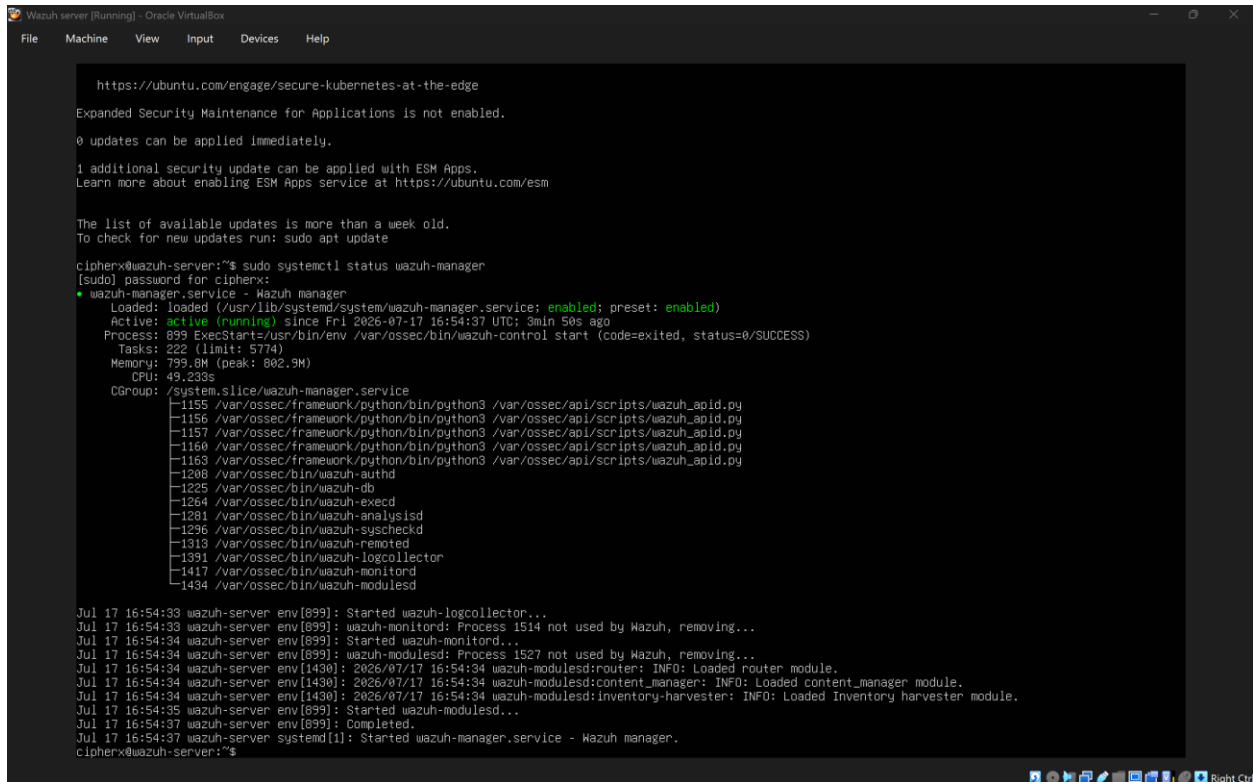
```
sudo systemctl status wazuh-manager
```

```
sudo systemctl status wazuh-indexer
```

```
sudo systemctl status wazuh-dashboard
```

All three services should display:

Active: active (running)



```
https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

cipherx@wazuh-server:~$ sudo systemctl status wazuh-manager
[sudo] password for cipherx:
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-07-17 16:54:37 UTC; 3min 50s ago
   Process: 899 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
   Tasks: 222 (limit: 5774)
   Memory: 799.8M (peak: 802.9M)
   CPU: 49.233s
   CGroup: /system.slice/wazuh-manager.service
           └─1155 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─1156 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─1157 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─1158 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─1159 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─1208 /var/ossec/bin/wazuh-authd
                       └─1225 /var/ossec/bin/wazuh-db
                         └─1264 /var/ossec/bin/wazuh-execd
                           └─1281 /var/ossec/bin/wazuh-analysisd
                             └─1296 /var/ossec/bin/wazuh-syscheckd
                               └─1313 /var/ossec/bin/wazuh-remoted
                                 └─1391 /var/ossec/bin/wazuh-logcollector
                                   └─1417 /var/ossec/bin/wazuh-monitord
                                     └─1434 /var/ossec/bin/wazuh-modulesd

Jul 17 16:54:33 wazuh-server env[899]: Started wazuh-logcollector...
Jul 17 16:54:33 wazuh-server env[899]: wazuh-monitord: Process 1514 not used by Wazuh, removing...
Jul 17 16:54:34 wazuh-server env[899]: Started wazuh-monitord...
Jul 17 16:54:34 wazuh-server env[899]: wazuh-modulesd: Process 1527 not used by Wazuh, removing...
Jul 17 16:54:34 wazuh-server env[1430]: 2026/07/17 16:54:34 wazuh-modulesd:router: INFO: Loaded router module.
Jul 17 16:54:34 wazuh-server env[1430]: 2026/07/17 16:54:34 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Jul 17 16:54:34 wazuh-server env[1430]: 2026/07/17 16:54:34 wazuh-modulesd:inventory-harvester: INFO: Loaded inventory harvester module.
Jul 17 16:54:35 wazuh-server env[899]: Started wazuh-modulesd...
Jul 17 16:54:37 wazuh-server env[899]: Completed.
Jul 17 16:54:37 wazuh-server systemd[1]: Started wazuh-manager.service - Wazuh manager.
cipherx@wazuh-server:~$
```

Figure 6.1 – Wazuh SIEM Services Running

Step 4 – Access the Wazuh Dashboard

Open your browser and navigate to:

<https://192.168.1.15> → Ubuntu-Server-IP

Log in using the administrator credentials generated during installation.

Step 5 – Install the Wazuh Agent (Windows 11)

Download and install the Windows Wazuh Agent.

During installation, provide:

Manager Address:

192.168.1.15

Agent Name:

Windows11-Agent

Step 6 – Start the Wazuh Agent

Open PowerShell as Administrator.

NET START WazuhSvc

or

Start-Service WazuhSvc

Step 7 – Verify Agent Status

Get-Service WazuhSvc

Expected output:

Status : Running

Step 8 – Verify Agent Connection (Ubuntu Server)

sudo /var/ossec/bin/agent_control -l

Expected output:

ID: 001

Name: Windows11-Agent

Status: Active

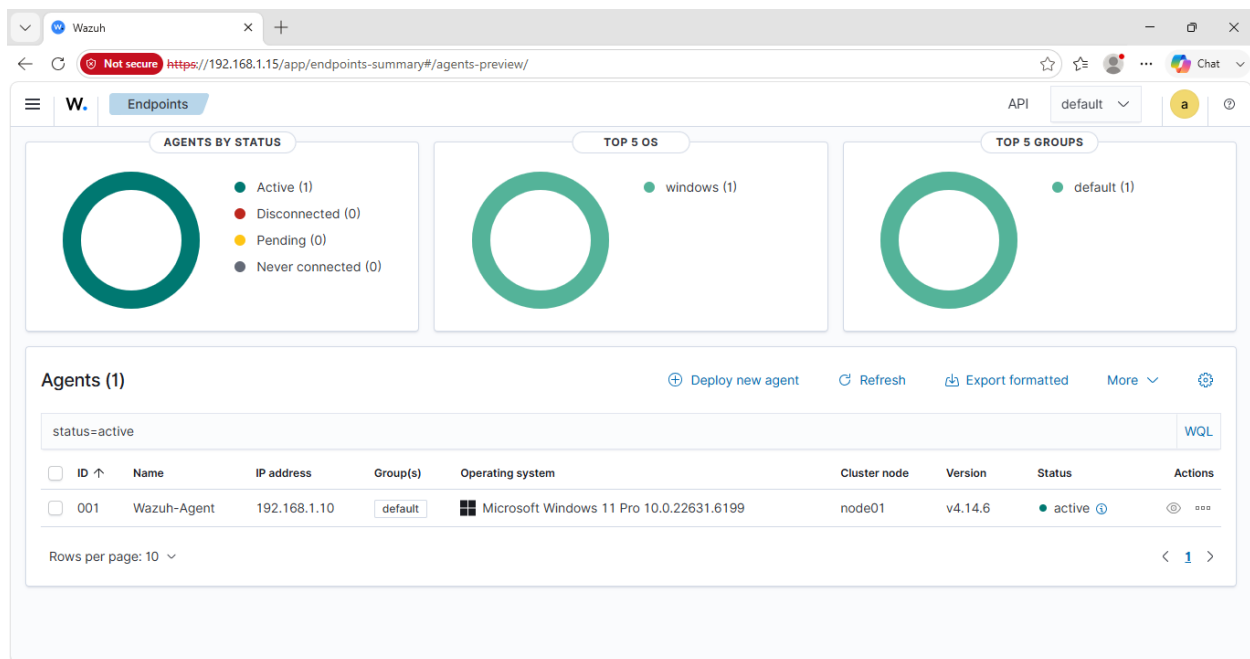


Figure 6.2 – Windows Agent Connected to Wazuh Dashboard

Verification

The successful deployment was confirmed by verifying that the Wazuh Manager, Indexer, and Dashboard services were running correctly on the Ubuntu Server. The Windows Wazuh Agent was then registered and connected successfully, enabling the endpoint to forward security events to the Wazuh SIEM platform for centralized monitoring and analysis.

6.2 Installing Sysmon & Integrating with Wazuh

Sysmon (System Monitor) was installed on the Windows 11 endpoint to enhance native Windows logging with detailed endpoint telemetry. After installation, Sysmon events were integrated with Wazuh, allowing advanced endpoint activities such as process creation, network connections, and registry modifications to be collected and analyzed through the Wazuh Dashboard.

Step 1 – Download Sysmon

Download the latest Sysmon package from the Microsoft Sysinternals Suite.

Step 2 – Install Sysmon

Open Command Prompt or PowerShell as Administrator and navigate to the Sysmon directory.

```
Sysmon64.exe -accepteula -i sysmonconfig.xml
```

Step 3 – Verify Sysmon Installation

```
Get-Service Sysmon64
```

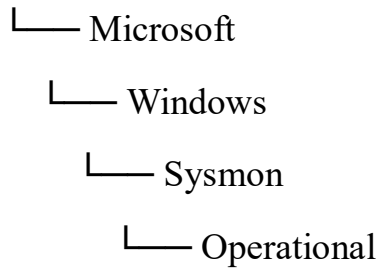
Expected output:

Status : Running

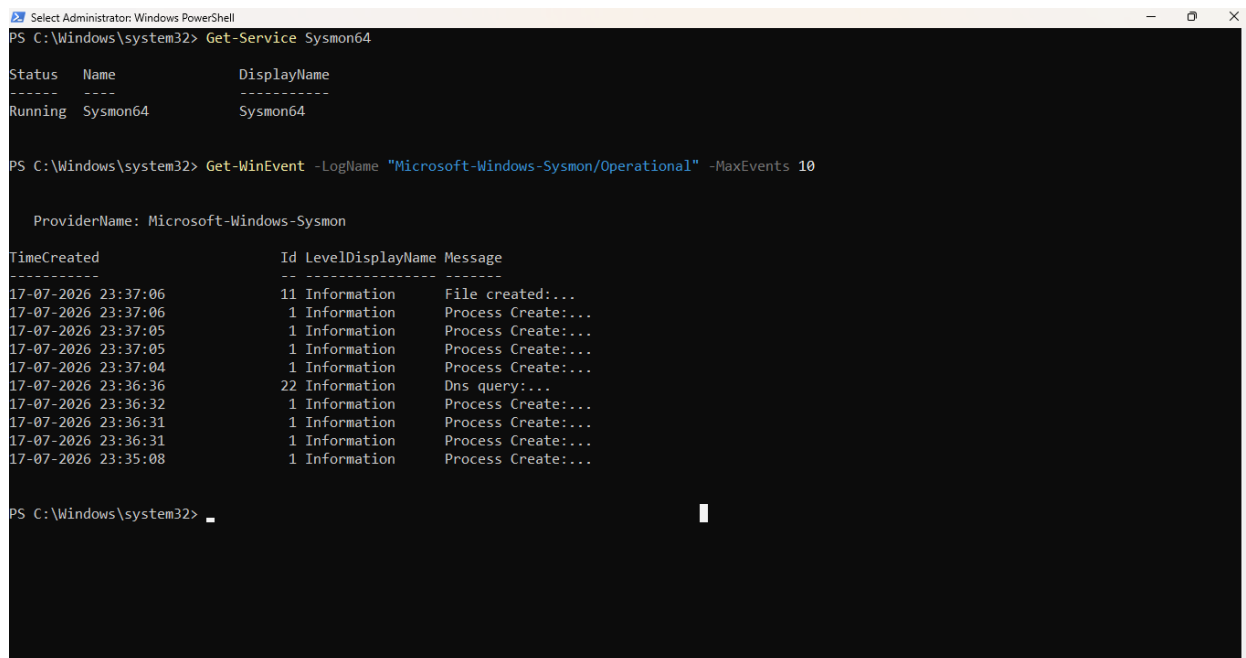
Step 4 – Verify Sysmon Logs

Open Event Viewer and navigate to:

Applications and Services Logs



Successful installation will generate Sysmon Event IDs such as Process Creation (Event ID 1), Network Connection (Event ID 3), and Process Termination (Event ID 5).



```
Select Administrator: Windows PowerShell
PS C:\Windows\system32> Get-Service Sysmon64

Status   Name      DisplayName
-----
Running  Sysmon64  Sysmon64

PS C:\Windows\system32> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 10

ProviderName: Microsoft-Windows-Sysmon

TimeCreated      Id LevelDisplayName Message
-----
17-07-2026 23:37:06 11 Information File created:...
17-07-2026 23:37:06 1 Information Process Create:...
17-07-2026 23:37:05 1 Information Process Create:...
17-07-2026 23:37:05 1 Information Process Create:...
17-07-2026 23:37:04 1 Information Process Create:...
17-07-2026 23:36:36 22 Information Dns query:...
17-07-2026 23:36:32 1 Information Process Create:...
17-07-2026 23:36:31 1 Information Process Create:...
17-07-2026 23:36:31 1 Information Process Create:...
17-07-2026 23:35:08 1 Information Process Create:...

PS C:\Windows\system32> _
```

Figure 6.3 – Sysmon Service Successfully Running

Step 5 – Integrate Sysmon with Wazuh

Configure the Wazuh Agent on the Windows endpoint to monitor the Sysmon Operational log by adding the following configuration to the agent configuration file.

<localfile>

<location>Microsoft-Windows-Sysmon/Operational</location>

<log_format>eventchannel</log_format>

</localfile>

Restart the Wazuh Agent after saving the configuration.

Restart-Service WazuhSvc

Step 6 – Verify Integration

Open the Wazuh Dashboard and search for:

event.provider: "Microsoft-Windows-Sysmon"

or

rule.groups : sysmon

The appearance of Sysmon events confirms that endpoint telemetry is being successfully collected and forwarded to the SIEM platform.

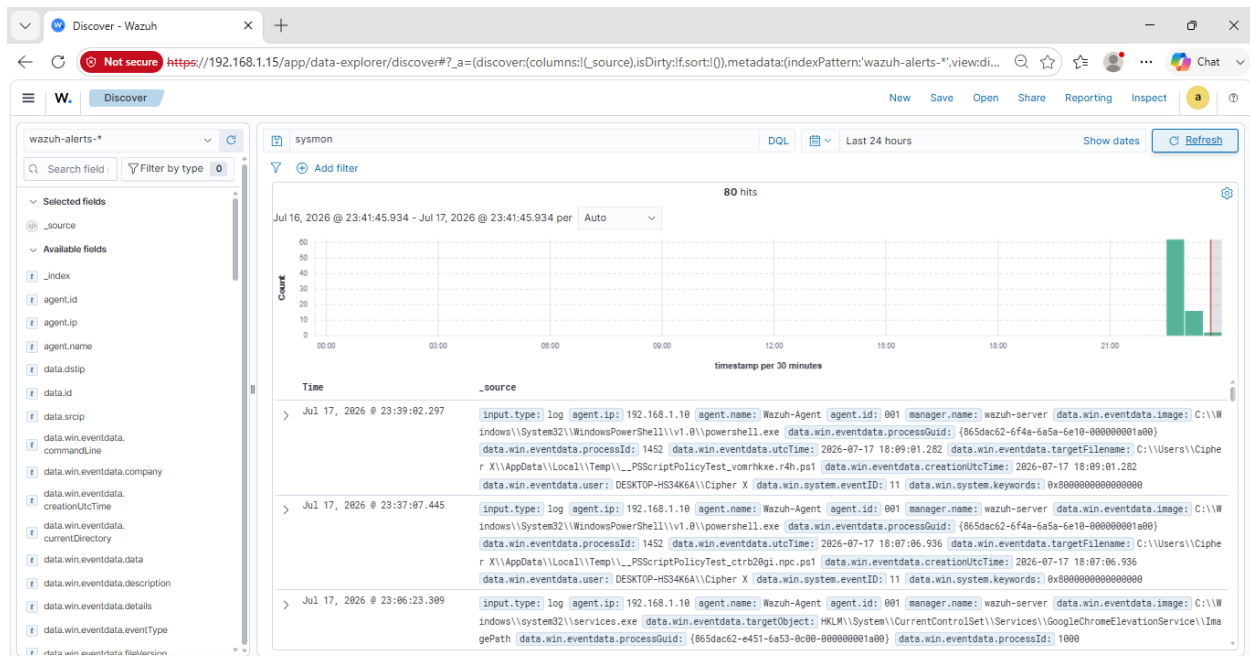


Figure 6.4 – Sysmon Events Displayed in Wazuh Dashboard

Verification

The Sysmon service was verified to be running successfully on the Windows endpoint, and operational events were confirmed through the Windows Event Viewer. After integrating Sysmon with the Wazuh Agent, the generated endpoint telemetry became visible within the Wazuh Dashboard, providing enhanced visibility into system activities beyond standard Windows Event Logs.

6.3 Installing Snort IDS & Integrating with Wazuh

Snort Intrusion Detection System (IDS) was deployed on the Windows 11 endpoint to monitor network traffic and identify suspicious or malicious activities. Snort was configured with community detection rules and integrated with Wazuh SIEM by forwarding alert logs, enabling centralized monitoring, investigation, and visualization of network-based attacks through the Wazuh Dashboard.

Step 1 – Install Snort IDS

Download the latest Snort package and install it on the Windows endpoint using the default installation options. After installation, verify that Snort has been successfully installed by checking its version.

```
snort -V
```

Expected Output

Snort Version 2.9.x

Step 2 – Configure Snort

Open the Snort configuration file and modify the required network variables according to the laboratory environment.

Configuration File:

```
C:\Snort\etc\snort.conf
```

Configure the HOME_NET variable with the Windows victim IP address.

```
var HOME_NET 192.168.1.10
```

Set the external network.

```
var EXTERNAL_NET any
```

Specify the location of Snort rules.

```
include $RULE_PATH/local.rules
```

Save the configuration file after completing the changes.

Step 3 – Verify Snort Configuration

Validate the configuration file before starting Snort to ensure there are no syntax errors.

```
snort -T -c C:\Snort\etc\snort.conf
```

Expected Output

Snort successfully validated the configuration.

Snort exiting

Step 4 – Start Snort Monitoring

After validation, start Snort in packet inspection mode using the active network interface.

```
snort -i 1 -A fast -c C:\Snort\etc\snort.conf -l C:\Snort\log
```

(Replace interface number if different.)

Check interface using snort -W

Snort begins inspecting incoming and outgoing network traffic while generating alert logs for suspicious activities.

Step 5 – Verify Alert Generation

Generate test network traffic such as ICMP requests or an Nmap scan from the attacker machine.

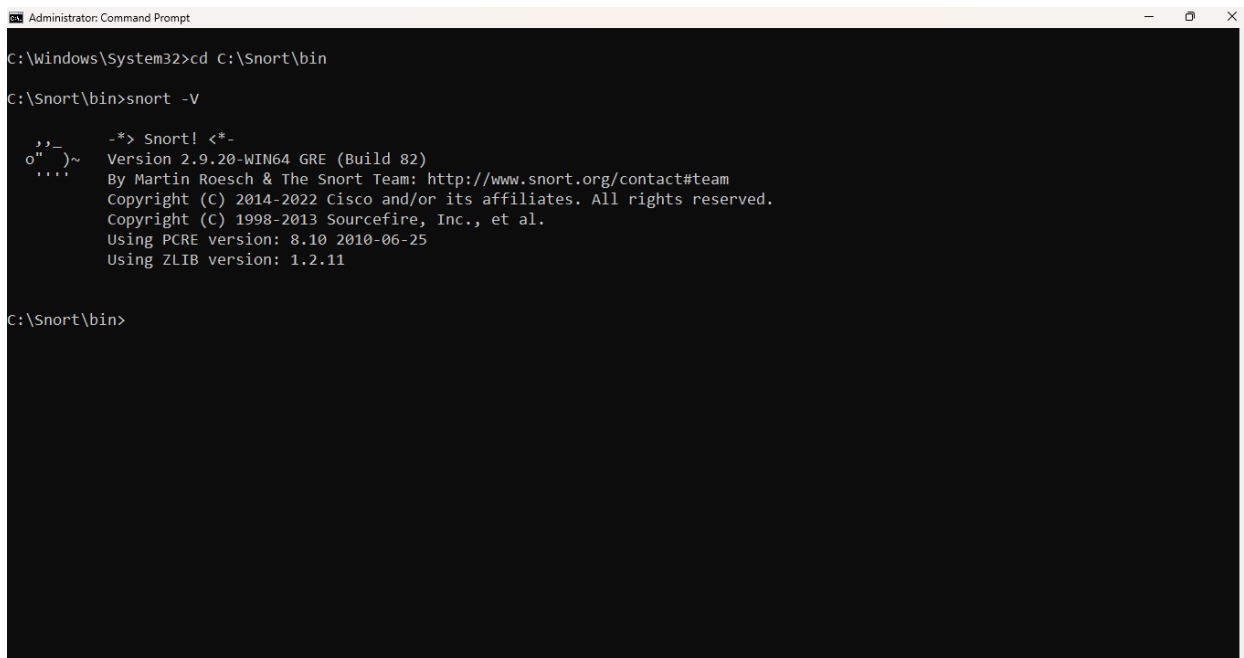
Verify that Snort creates alert logs inside the logging directory.

type C:\Snort\log\alert.ids

or

notepad C:\Snort\log\alert.ids

Successful detections confirm that Snort is functioning correctly.



```
Administrator: Command Prompt
C:\Windows\System32>cd C:\Snort\bin
C:\Snort\bin>snort -V

  ,,-  -*> Snort! <*-
 o"  )~ Version 2.9.20-WIN64 GRE (Build 82)
  '    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
        Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
        Copyright (C) 1998-2013 Sourcefire, Inc., et al.
        Using PCRE version: 8.10 2010-06-25
        Using ZLIB version: 1.2.11

C:\Snort\bin>
```

Figure 6.5 – Snort Running Successfully

Step 6 – Integrate Snort with Wazuh

Configure the Wazuh Agent to monitor the Snort alert log by adding the following entry to the Wazuh Agent configuration file.

```
<localfile>  
  <location>C:\Snort\log\alert.ids</location>  
  <log_format>syslog</log_format>  
</localfile>
```

After saving the configuration, restart the Wazuh Agent.

Restart-Service WazuhSvc

Step 7 – Verify Wazuh Integration

Open the Wazuh Dashboard and navigate to **Threat Hunting**.

Generate a test attack from the Kali Linux attacker machine and confirm that Snort alerts are forwarded successfully to Wazuh.

Successful integration is verified when Snort alerts appear in the Wazuh Dashboard with associated rule information, timestamps, source IP addresses, and alert descriptions.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 17, 2026 @ 23:30:01.277	Wazuh-Agent	Multiple IDS alerts for same id.	10	20152
Jul 17, 2026 @ 23:30:01.277	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:30:01.256	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:58.334	Wazuh-Agent	Software protection service scheduled successfully.	3	60642
Jul 17, 2026 @ 23:29:53.229	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:53.229	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:53.224	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:52.870	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:52.856	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:52.841	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:52.825	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:52.809	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:52.803	Wazuh-Agent	IDS event.	6	20101
Jul 17, 2026 @ 23:29:52.803	Wazuh-Agent	First time this IDS alert is generated.	8	20100

Figure 6.6 – Snort Alerts in Wazuh Dashboard

Verification

Snort IDS was successfully deployed to inspect network traffic and detect malicious activities. Alert logs were generated during attack simulations and forwarded to the Wazuh SIEM platform through the Wazuh Agent. The integration enabled centralized monitoring, threat investigation, and real-time visualization of IDS alerts within the Wazuh Dashboard.

7. ATTACK SIMULATIONS

This chapter demonstrates controlled cyberattack simulations executed from the Kali Linux attacker machine against the Windows 11 endpoint within the laboratory environment. The objective of these simulations is to validate the effectiveness of the deployed security controls by generating realistic attack traffic and verifying successful detection through Snort IDS and Wazuh SIEM. Each attack represents a common phase of the cyber kill chain, allowing the monitoring platform to detect, collect, and correlate security events for centralized investigation.

7.1 ICMP RECONNAISSANCE DETECTION

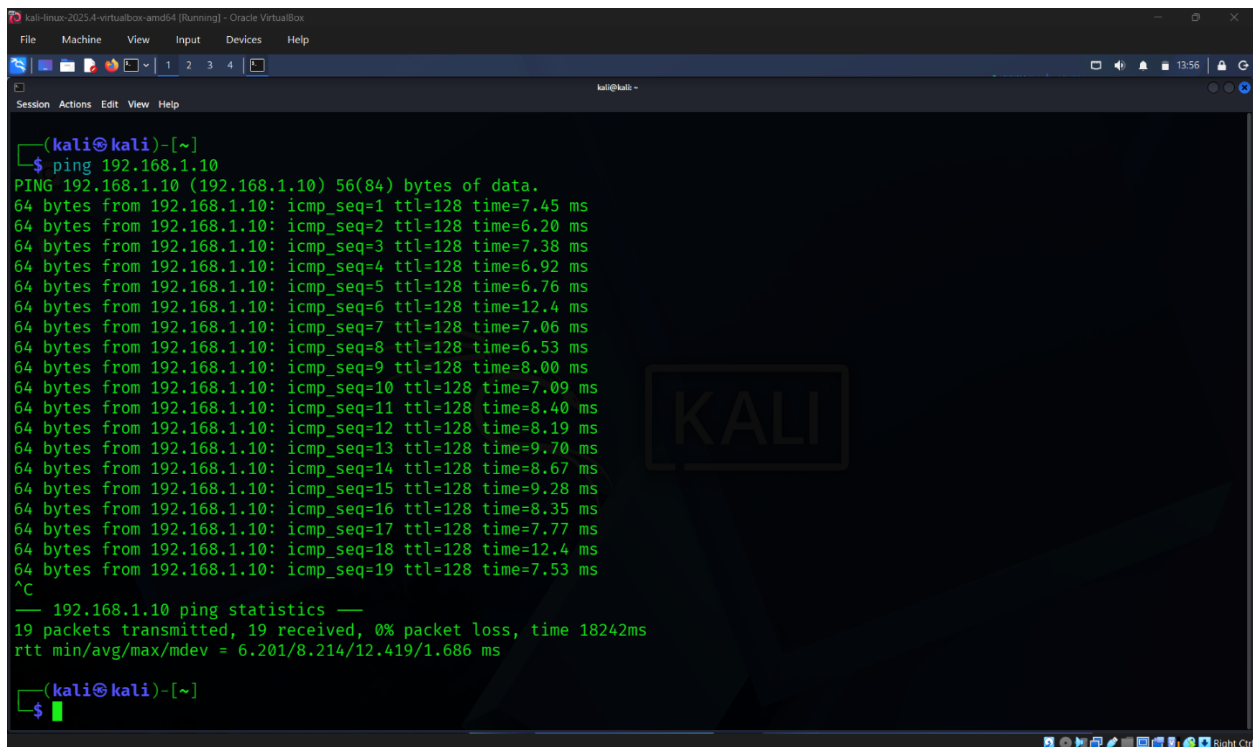
Attack Overview

ICMP Echo Requests (Ping) are commonly used during the reconnaissance phase of a cyberattack to determine whether a target host is reachable on the network. Although ICMP itself is a legitimate network protocol, repeated ping requests can indicate the initial stage of an attacker identifying live systems before conducting further enumeration or exploitation.

Attack Command

The following command was executed from the Kali Linux attacker machine.

ping 192.168.1.10 → Victim Ip



```
(kali@kali)-[~]
$ ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=128 time=7.45 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=128 time=6.20 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=128 time=7.38 ms
64 bytes from 192.168.1.10: icmp_seq=4 ttl=128 time=6.92 ms
64 bytes from 192.168.1.10: icmp_seq=5 ttl=128 time=6.76 ms
64 bytes from 192.168.1.10: icmp_seq=6 ttl=128 time=12.4 ms
64 bytes from 192.168.1.10: icmp_seq=7 ttl=128 time=7.06 ms
64 bytes from 192.168.1.10: icmp_seq=8 ttl=128 time=6.53 ms
64 bytes from 192.168.1.10: icmp_seq=9 ttl=128 time=8.00 ms
64 bytes from 192.168.1.10: icmp_seq=10 ttl=128 time=7.09 ms
64 bytes from 192.168.1.10: icmp_seq=11 ttl=128 time=8.40 ms
64 bytes from 192.168.1.10: icmp_seq=12 ttl=128 time=8.19 ms
64 bytes from 192.168.1.10: icmp_seq=13 ttl=128 time=9.70 ms
64 bytes from 192.168.1.10: icmp_seq=14 ttl=128 time=8.67 ms
64 bytes from 192.168.1.10: icmp_seq=15 ttl=128 time=9.28 ms
64 bytes from 192.168.1.10: icmp_seq=16 ttl=128 time=8.35 ms
64 bytes from 192.168.1.10: icmp_seq=17 ttl=128 time=7.77 ms
64 bytes from 192.168.1.10: icmp_seq=18 ttl=128 time=12.4 ms
64 bytes from 192.168.1.10: icmp_seq=19 ttl=128 time=7.53 ms
^C
  -- 192.168.1.10 ping statistics --
19 packets transmitted, 19 received, 0% packet loss, time 18242ms
rtt min/avg/max/mdev = 6.201/8.214/12.419/1.686 ms
(kali@kali)-[~]
$
```

Figure 7.1 – Attack Executed from Kali Linux

Detection

Snort IDS successfully detected the incoming ICMP Echo Requests and generated an alert indicating reconnaissance activity. The generated IDS alert was forwarded to the Wazuh SIEM platform, where it was centralized for investigation and analysis.

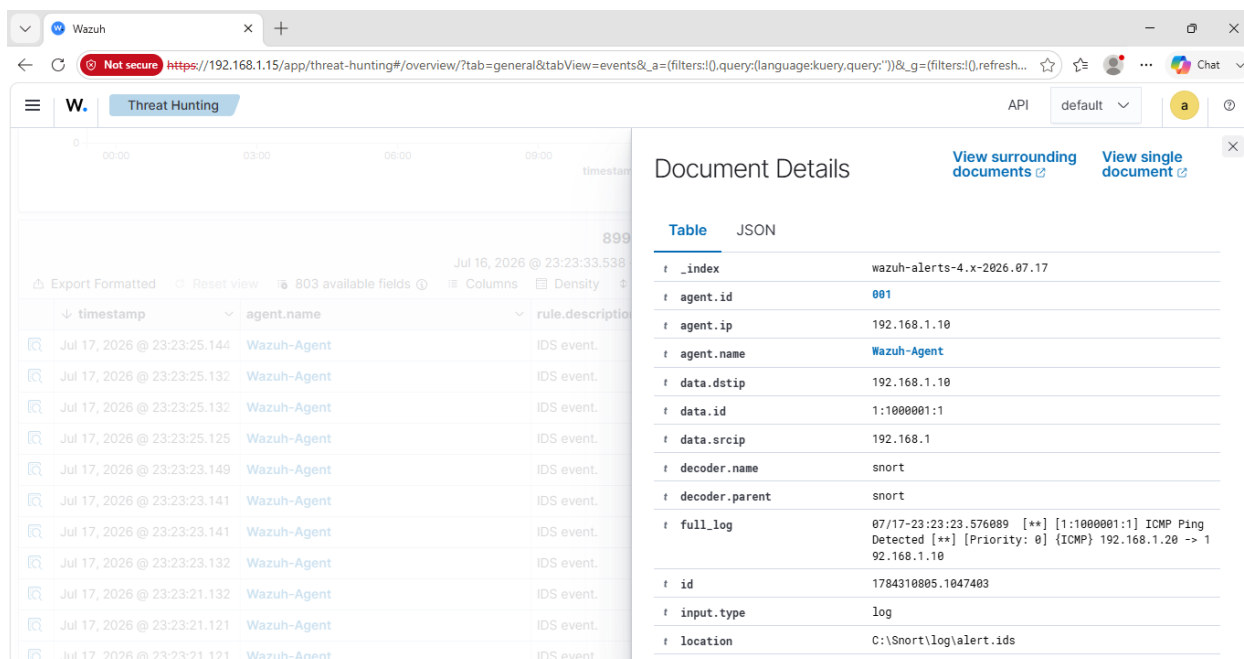


Figure 7.2 – ICMP Ping Detection in Wazuh Dashboard

Analysis

The successful detection of ICMP reconnaissance demonstrates that the monitoring environment can identify the earliest stage of an attack before any exploitation occurs. Early detection allows security analysts to investigate suspicious hosts, identify potential adversaries, and implement preventive measures before the attack progresses further.

MITRE ATT&CK Technique: T1018 – Remote System Discovery

Tactic: Reconnaissance (TA0043)

7.2 TCP SYN / Nmap Port Scan Detection

Attack Overview

After identifying that the target host was reachable, the attacker performed a TCP SYN port scan using Nmap to enumerate open ports and discover available network services. TCP SYN scanning, commonly known as a **half-open scan**, is a stealth reconnaissance technique that sends TCP SYN packets without completing the full three-way handshake. This method allows attackers to identify accessible services while minimizing the likelihood of detection by the target system.

Attack Command

The following command was executed from the Kali Linux attacker machine to perform a TCP SYN scan against the Windows 11 endpoint.

```
nmap -sS 192.168.1.10 or
```

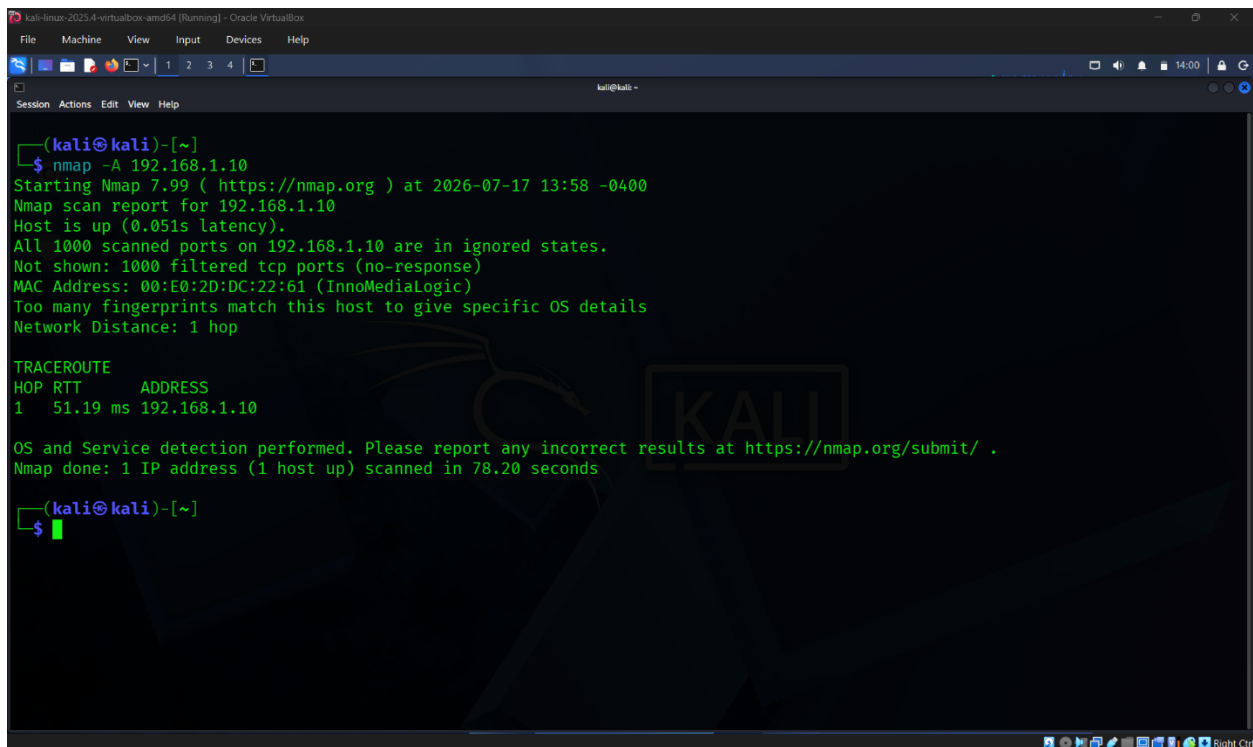
```
nmap -Pn -sS 192.168.1.10 or
```

```
nmap -A 192.168.1.10
```

-sS → TCP SYN (Half-open) Scan

-Pn -sS → SYN Scan without host discovery (useful if ICMP is blocked)

-A → Aggressive Scan (OS detection, version detection, traceroute, default scripts)



```
(kali@kali)-[~]
$ nmap -A 192.168.1.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-17 13:58 -0400
Nmap scan report for 192.168.1.10
Host is up (0.051s latency).
All 1000 scanned ports on 192.168.1.10 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:E0:2D:DC:22:61 (InnoMediaLogic)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 51.19 ms 192.168.1.10

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 78.20 seconds

(kali@kali)-[~]
$
```

Figure 7.3 – Attack Executed from Kali Linux

Detection

Snort IDS identified abnormal TCP behavior by detecting incomplete TCP three-way handshakes generated during the SYN scan. The IDS classified the activity as Potentially Bad Traffic and generated alerts indicating suspicious reconnaissance activity. These alerts were forwarded to the Wazuh SIEM platform, enabling centralized monitoring and investigation.

The screenshot displays the Wazuh Threat Hunting interface. On the left, a table lists events with columns for timestamp, agent name, and rule description. The right pane shows 'Document Details' for a selected event, including fields like _index, agent.id, agent.ip, agent.name, data.dstip, data.id, data.srcip, decoder.name, decoder.parent, full_log, id, input.type, and location.

timestamp	agent.name	rule.description
Jul 17, 2026 @ 23:30:01.277	Wazuh-Agent	Multiple IDS alerts
Jul 17, 2026 @ 23:30:01.277	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:30:01.256	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:58.334	Wazuh-Agent	Software protection
Jul 17, 2026 @ 23:29:53.229	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:53.229	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:53.224	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:52.870	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:52.856	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:52.841	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:52.825	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:52.809	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:52.803	Wazuh-Agent	IDS event.
Jul 17, 2026 @ 23:29:52.803	Wazuh-Agent	First time this IP
Jul 17, 2026 @ 23:29:52.803	Wazuh-Agent	IDS event.

Field	Value
_index	wazuh-alerts-4.x-2026.07.17
agent.id	001
agent.ip	192.168.1.10
agent.name	Wazuh-Agent
data.dstip	192.168.1.10:41489
data.id	129:20:1
data.srcip	192.168.1.20
decoder.name	snort
decoder.parent	snort
full_log	07/17-23:29:51.173143 [++] [129:20:1] TCP session without 3-way handshake [**] [Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 192.168.1.20:34213 -> 192.168.1.10:41489
id	1784311192.1097416
input.type	log
location	C:\Snort\log\alert.ids

Figure 7.4 – TCP SYN Scan Detected in Wazuh Dashboard

Analysis

TCP SYN scanning is one of the most common reconnaissance techniques used by attackers before exploitation. By identifying this behavior, the implemented IDS successfully detected an attempt to enumerate network services without establishing legitimate TCP connections. Early identification of reconnaissance activities enables security analysts to investigate suspicious hosts, monitor attacker behavior, and implement defensive measures before an attack progresses to the exploitation stage.

MITRE ATT&CK Technique: T1046 – Network Service Discovery

Tactic: Reconnaissance

7.3 SSH Brute Force Detection (Hydra)

Attack Overview

Following reconnaissance and service enumeration, the attacker attempted to gain unauthorized access to the target system using **Hydra**, a password-cracking tool capable of performing automated brute-force attacks against network services. In this simulation, Hydra targeted the SSH service running on the Windows 11 endpoint by repeatedly attempting different username and password combinations.

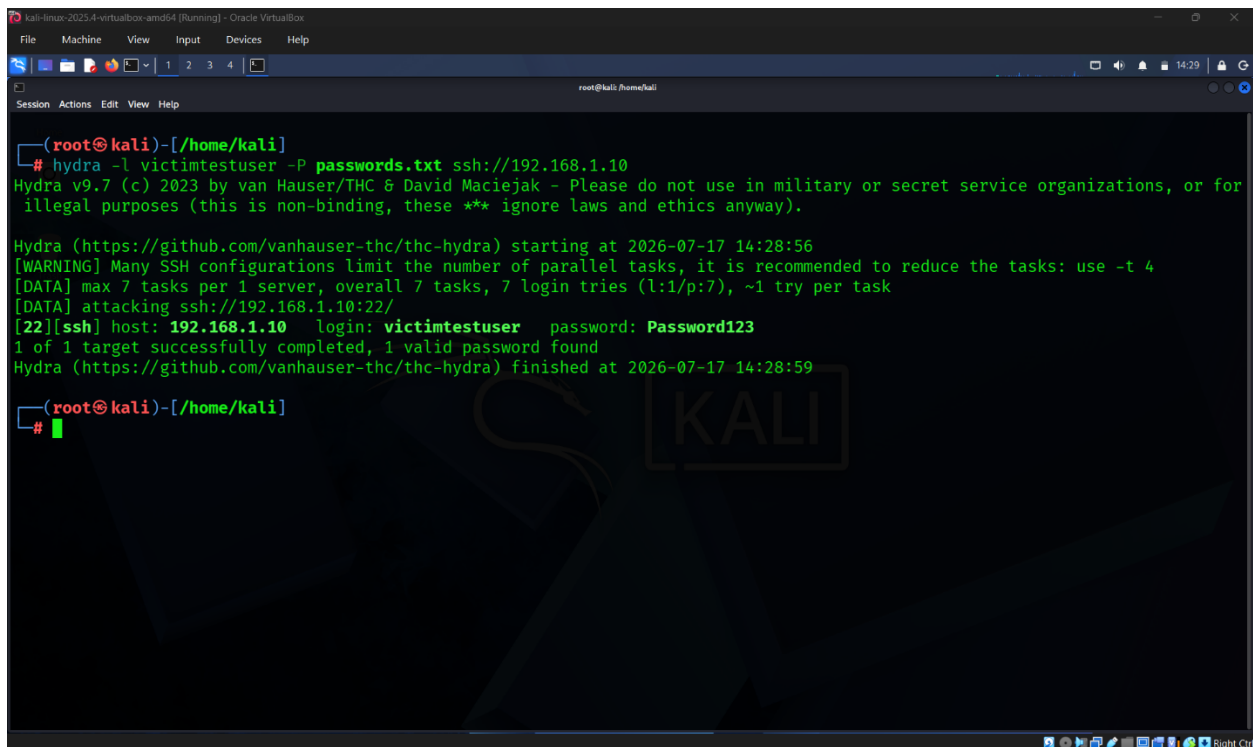
Attack Command

The following command was executed from the Kali Linux attacker machine to perform an SSH brute-force attack.

```
hydra -l victimtestuser -P passwords.txt ssh://192.168.1.10
```

Alternative Commands:

```
hydra -L users.txt -P passwords.txt ssh://192.168.1.10
```



```
(root@kali)-[/home/kali]
# hydra -l victimtestuser -P passwords.txt ssh://192.168.1.10
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for
illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-17 14:28:56
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 7 tasks per 1 server, overall 7 tasks, 7 login tries (l:1/p:7), ~1 try per task
[DATA] attacking ssh://192.168.1.10:22/
[22][ssh] host: 192.168.1.10  login: victimtestuser  password: Password123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-17 14:28:59

(root@kali)-[/home/kali]
#
```

Figure 7.5 – Attack Executed from Kali Linux

Detection

During the attack simulation, multiple SSH authentication attempts generated suspicious network activity, which was successfully detected by Snort IDS. The generated alerts were forwarded to the Wazuh SIEM platform, allowing centralized monitoring and investigation of the attempted credential attack. Even though the authentication was unsuccessful, the repeated connection attempts clearly indicated brute-force behavior originating from the attacker machine.

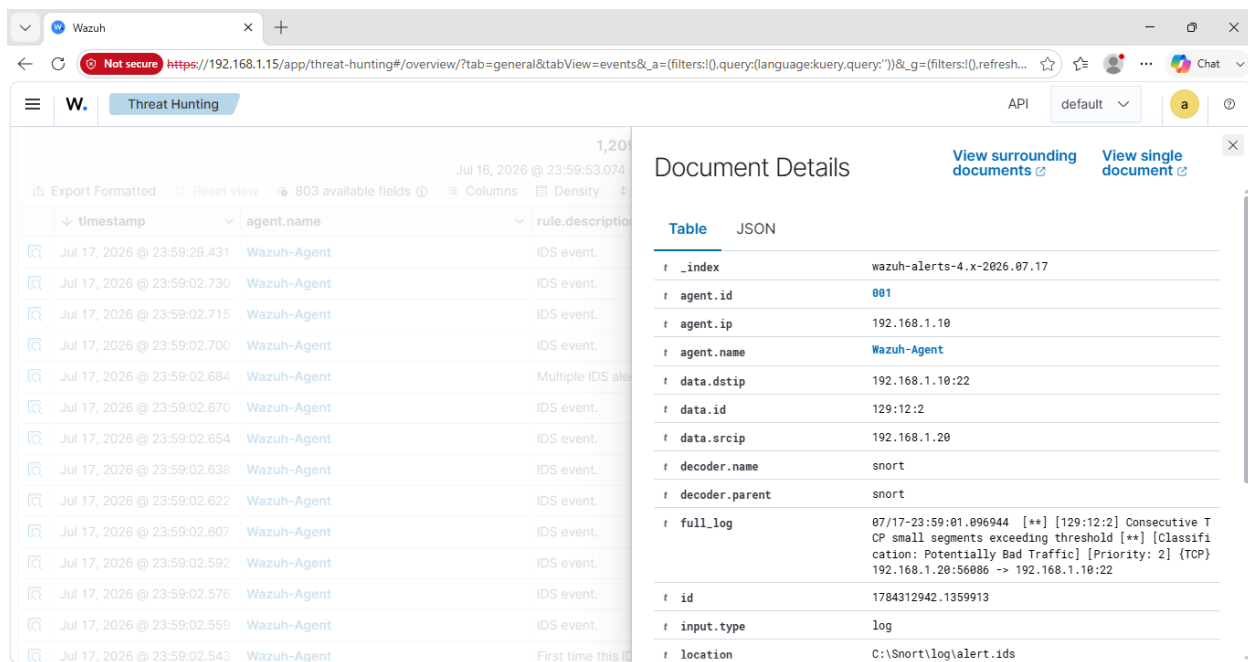


Figure 7.6 – SSH Brute Force Activity Detected in Wazuh Dashboard

Analysis

SSH brute-force attacks are commonly used by attackers to gain unauthorized access by systematically testing multiple username and password combinations. Detecting repeated authentication attempts enables security analysts to identify credential attacks at an early stage, correlate them with previous reconnaissance activities, and implement defensive measures such as account lockout policies, rate limiting, multi-factor authentication (MFA), and IP-based access restrictions.

MITRE ATT&CK Technique: T1110 – Brute Force

Tactic: Credential Access (TA0006)

8. CONCLUSION

This project successfully demonstrated the design and implementation of an Enterprise Threat Detection & Monitoring Laboratory using open-source security technologies. The environment integrated Wazuh SIEM, Snort IDS, Sysmon, and Windows Event Logging to provide centralized monitoring, event correlation, and security alert visualization within a controlled virtual environment.

Three attack simulations—ICMP reconnaissance, TCP SYN/Nmap port scanning, and SSH brute-force attempts—were successfully executed from the Kali Linux attacker machine and detected through the deployed monitoring infrastructure. The generated security events were collected, analyzed, and investigated using the Wazuh Dashboard, demonstrating the complete workflow of attack detection and incident analysis.

The project provided practical experience in deploying security monitoring solutions, integrating multiple security tools, and performing basic SOC-style investigations. It also strengthened knowledge of network monitoring, endpoint visibility, intrusion detection, and log analysis, making this laboratory a strong foundation for future cybersecurity research and enterprise security operations.

9. APPENDIX A – AUTHOR INFORMATION

A.1 Project Description.

Project Title: Enterprise Threat Detection & Monitoring Laboratory

Project Type: Academic & Personal Portfolio Project

Project Duration: July 2026

Prepared By: Rahul Reghu Rajan

Educational Qualification:

- Bachelor of Computer Applications (BCA)
- Advanced Diploma in Cyber Defence (ADCD)
- Certified Ethical Hacker (CEH v13)

Professional Interests:

- Cybersecurity
- Security Operations
- Ethical Hacking
- Security Analysis
- Governance, Risk & Compliance (GRC)

Project Overview:

This project demonstrates the implementation of an enterprise-grade security monitoring laboratory using Wazuh SIEM, Snort IDS, Sysmon, Windows Event Logging, and Kali Linux. The lab was designed to simulate real-world cyber attacks, detect malicious activities, investigate security alerts, and understand the workflow followed by Security Operations Centers (SOC). The project showcases practical knowledge in security monitoring, intrusion detection, log analysis, and incident investigation within a virtualized enterprise environment.

A.2 Contact Information

Author:

Rahul Reghu Rajan

Email:

rahulreghurajan@gmail.com

Phone:

(+91) 9385718653

Portfolio:

<https://rahulhub.vercel.app>

GitHub:

[RAHUL-REGHU-RAJAN \(Rahul Reghu Rajan\)](#)

LinkedIn:

[Rahul Reghu Rajan | LinkedIn](#)

10.REFERENCES

1. Wazuh Documentation. *Wazuh Documentation*. Available at: <https://documentation.wazuh.com/>
2. Wazuh. *Wazuh Installation Guide*. Available at: <https://documentation.wazuh.com/current/installation-guide/index.html>
3. Snort Project. *Snort 2 Documentation*. Available at: <https://www.snort.org/documents>
4. Cisco. *Snort Open Source Intrusion Prevention System*. Available at: <https://www.snort.org/>
5. Microsoft. *Sysmon - Windows Sysinternals*. Available at: <https://learn.microsoft.com/sysinternals/downloads/sysmon>
6. Microsoft. *Windows Event Logging*. Available at: <https://learn.microsoft.com/windows/win32/eventlog/event-logging>
7. Nmap Project. *Nmap Reference Guide*. Available at: <https://nmap.org/book/>
8. THC Hydra. *Hydra GitHub Repository*. Available at: <https://github.com/vanhauser-thc/thc-hydra>
9. MITRE Corporation. *MITRE ATT&CK® Framework*. Available at: <https://attack.mitre.org/>
10. Oracle. *Oracle VM VirtualBox Documentation*. Available at: <https://www.virtualbox.org/wiki/Documentation>
11. Kali Linux Documentation. *Official Kali Linux Documentation*. Available at: <https://www.kali.org/docs/>
12. Ubuntu Documentation. *Ubuntu Server Guide*. Available at: <https://ubuntu.com/server/docs>